SVM Notes - English - Atlas Skilltech

Nov 7, 2026 . @Razeen

SVM is a powerful supervised learning algorithm that finds the optimal boundary between classes by maximizing the margin.

1. Introduction to Support Vector Machines

SVM is a **supervised machine learning algorithm** used for both classification and regression. Its core idea: find the hyperplane that best separates classes with the **maximum possible margin**.

Intuition:

> Given red and blue marbles on a table, draw the line that keeps them farthest apart — so new marbles land on the correct side most reliably.

Real-World Examples:

Application	Features	Output	Industry
Email spam filter	TF-IDF text vectors	Spam / Not Spam	Tech
Cancer diagnosis	Tumor measurements	Malignant / Benign	Healthcare
Face recognition	Pixel values	Person ID	Security
Fraud detection	Transaction features	Fraud / Legit	Finance
Handwriting OCR	Pixel intensities	Character label	Education

SVM vs Other Algorithms:

Property	SVM	Logistic Regression	KNN
Training speed	Slow	Fast	None
Prediction speed	Fast	Fast	Slow
Non-linear data	Kernel trick	Polynomial features	Distance-based
High-dim performance	Excellent	Good	Poor

Property	SVM	Logistic Regression	KNN
Memory footprint	Small (SVs only)	Small	Large (all data)

When to use SVM:

- Medium-sized datasets (not millions of rows)
- High-dimensional feature spaces (images, text)
- Clear class separation required
- Both linear and non-linear boundaries needed

2. Hyperplanes and Margins

Hyperplane Definition

A hyperplane separates the feature space into two half-spaces:

- **2D:** a line `ax + by + c = 0`
- **3D:** a plane
- **nD:** an (n-1)-dimensional affine subspace

Margin and Optimization

The **margin** is the perpendicular distance between the hyperplane and the nearest data points on each side.

```
\text{Margin} = \frac{2}{\|w\|}
```

SVM maximizes this margin. Equivalently, it minimizes `||w||`:

```
\min_{w, b} \frac{1}{2} \|w\|^2 \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geq 1
```

Support Vectors

Support vectors are the data points satisfying `y_i(w·x_i + b) = 1`, lying exactly on the margin boundaries. They are the **only points that define the hyperplane** — all other training points are irrelevant to the final model.

Soft Margin (C Parameter)

Real-world data is rarely perfectly separable. The soft-margin formulation introduces slack variables $\xi_i \geq 0$:

$$\min_{w, b, \xi} \frac{1}{2}\|w\|^2 + C \sum_{i=1}^{n} \xi_i$$

Subject to:

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

C Value	Behavior	Risk
Very large (e.g. 1000)	Hard margin, zero tolerance	Overfitting
Moderate (e.g. 1-10)	Balanced	Good generalization
Very small (e.g. 0.001)	Wide margin, high tolerance	Underfitting

Analogy: C is how strict the classifier is. High C = perfectionist (memorizes training noise). Low C = too lenient (ignores useful patterns).

3. Kernel Intuition

The Non-Linear Problem

Many real-world datasets are **not linearly separable** — no straight hyperplane can correctly classify them. Classic examples: concentric circles, XOR patterns, interleaved spirals.

The Key Idea

Map data to a higher-dimensional space where it **becomes** linearly separable, then apply a linear SVM there.

Marble Analogy:

> Blue marbles inside a circle, red marbles outside — no line separates them on the flat table. Flip the table slightly (add a z-dimension): blue marbles rise up, red stay low. Now a flat plane perfectly separates them. The kernel trick does this transformation implicitly — without computing the actual high-dimensional coordinates.

Why Implicit Computation?

Explicitly computing the transformation `Φ(x)` for high dimensions is computationally expensive or even infinite-dimensional (RBF). The kernel trick evaluates `K(x_i, x_j) = Φ(x_i) · Φ(x_j)` directly in the original space — same mathematical result, fraction of the cost.

Four Kernels Compared

Kernel	Best For	Boundary Shape	Real Example
Linear	High-dim text, linearly separable	Hyperplane	Spam filter, news categorization
RBF	General non-linear, circular	Radial / circular	Medical imaging, face ID
Polynomial	Computer vision, structured data	Polynomial curve	OCR, image classification
Sigmoid	NLP, neural net-like behavior	S-shaped	Sentiment analysis

Gamma (γ) in RBF — The Bandwidth

> Think of γ as how far a training point's "influence" reaches. High γ: each point influences only its immediate neighborhood → jagged boundary → overfit. Low γ: each point influences a wide area → smooth boundary → underfit. `gamma='scale'` (default): `1 / (n_features * X.var())` — auto-scales to your data.

4. Kernel Mathematics

Linear Kernel

$$K(x_i, x_j) = x_i \cdot x_j = \sum_{k=1}^{d} x_{ik} \cdot x_{jk}$$

Simple dot product in the original space. No transformation. Fast for linearly separable, high-dimensional data.

RBF (Radial Basis Function / Gaussian) Kernel

$$K(x_i, x_j) = \exp\left(-\gamma \|x_i - x_j\|^2\right), \quad \gamma > 0$$

When points are close: `||xi-xj||` small → K near 1. When far apart: K near 0. Equivalent to an infinite-dimensional dot product — the most versatile kernel.

Polynomial Kernel

$$K(x_i, x_j) = (\gamma \cdot x_i \cdot x_j + r)^d$$

Parameters: `d` = degree, `r` = coef0. Degree 2 gives quadratic boundaries; degree 3 cubic, and so on.

Sigmoid Kernel

$$K(x_i, x_j) = \tanh(\gamma \cdot x_i \cdot x_j + r)$$

Mimics a two-layer neural network's activation. Not always positive-definite — use with caution.

Dual Formulation and Decision Function

The SVM dual problem replaces `w` with the support vector expansion:

$$\max_{\alpha} \sum_{i} \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$

Subject to: $0 \leqslant \alpha_i \leqslant C$ and $\Sigma\ \alpha_i\ y_i = 0$

Final decision function:

```
f(x) = \text{sign}\left(\sum_{i \in SV} \alpha_i y_i K(x_i, x) + b\right)
```

Parameter Reference

Parameter	Kernel	Effect	Recommended Range
C	All	Margin softness / regularization	0.001 – 1000 (log scale)
gamma	RBF, Poly, Sigmoid	Influence radius / boundary complexity	'scale', 0.001 – 10
degree	Polynomial	Polynomial order	2, 3, 4
coef0	Polynomial, Sigmoid	Kernel offset	0 – 1

5. SVM Classification Implementation

Complete Pipeline on Breast Cancer Dataset

We use the **Breast Cancer Wisconsin Dataset** — a real medical dataset with 569 samples and 30 features. The goal: classify tumors as **Malignant (cancerous)** or **Benign (non-cancerous)**.

Why this dataset? It perfectly demonstrates why we choose Recall over Precision in medical applications — a False Negative (missed cancer) is far more dangerous than a False Positive.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay
import seaborn as sns
import matplotlib.pyplot as plt

# Step 1: Load Data
data = load_breast_cancer()
```

```
X, y = data.data, data.target
# y: 0 = Malignant, 1 = Benign

# Step 2: Split Data (stratified to maintain class ratio)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Step 3: Scale Features (CRITICAL for SVM!)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)  # only transform, never fit!

# Step 4: Train Model
model = SVC(kernel='rbf', C=10.0, gamma='scale', probability=True)
model.fit(X_train_scaled, y_train)

# Step 5: Predict
y_pred = model.predict(X_test_scaled)

# Step 6: Evaluate
print(classification_report(y_test, y_pred, target_names=data.target_names))
```

Confusion Matrix — Two Methods

Method 1 (Classic — Seaborn Heatmap):

```
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=data.target_names,
            yticklabels=data.target_names)
plt.title('Confusion Matrix — SVM (Seaborn)')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

Method 2 (Modern — ConfusionMatrixDisplay):

```
from sklearn.metrics import ConfusionMatrixDisplay
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=data.target_names)
disp.plot(cmap='Blues')
plt.title('Confusion Matrix — SVM (sklearn Display)')
plt.show()
```

Understanding Metrics in Medical Context

Metric	Formula	Meaning	Priority
Precision	TP / (TP + FP)	Of predicted positives, how many are actually positive	Secondary
Recall (Sensitivity)	TP / (TP + FN)	Of actual positives, how many did we catch	**PRIMARY**
F1-Score	2 × (P × R) / (P + R)	Harmonic mean of Precision and Recall	Balance metric
Specificity	TN / (TN + FP)	Of actual negatives, how many correctly identified	

Why Recall matters most here: A False Negative = patient has cancer but we say "benign" → they go untreated → life-threatening. A False Positive = healthy patient gets extra tests → inconvenient, but not fatal.

ROC Curve — Two Methods

Method 1 (Classic — Manual):

```
from sklearn.metrics import roc_curve, auc
y_proba = model.predict_proba(X_test_scaled)[:, 1]
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.3f}')
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate (Recall)')
plt.title('ROC Curve — Manual Method')
plt.legend()
plt.show()
```

Method 2 (Modern — RocCurveDisplay):

```
from sklearn.metrics import RocCurveDisplay
RocCurveDisplay.from_estimator(model, X_test_scaled, y_test)
plt.title('ROC Curve — RocCurveDisplay (sklearn)')
plt.show()
```

GridSearchCV — Systematic Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 'auto', 0.001, 0.01]
}

# scoring='recall' because we care about catching all cancers
grid_search = GridSearchCV(
    SVC(kernel='rbf', probability=True),
    param_grid,
    scoring='recall',
    cv=5,
    verbose=1
)
grid_search.fit(X_train_scaled, y_train)
```

```
print(f'Best Parameters: {grid_search.best_params_}')
print(f'Best Recall Score: {grid_search.best_score_:.4f}')
best_model = grid_search.best_estimator_
```

Key Insight: We use `scoring='recall'` instead of `scoring='accuracy'` because in cancer detection, missing a cancer patient (False Negative) is far worse than a false alarm.

6. Support Vector Regression (SVR)

Core Concept: The Epsilon Tube

While SVM Classification finds a hyperplane that **separates** classes, SVR finds a hyperplane that **fits** continuous data within a tolerance band called the **epsilon tube (e-tube)**.

Key Insight: SVR says: "I don't care about small errors. Any prediction within epsilon of the true value is considered correct."

Analogy: Imagine you're drawing a highway through scattered data points. You draw two parallel lanes (the epsilon-tube). Points inside the lane are fine — no penalty. Points outside get penalized proportionally to how far they stick out.

Mathematical Formulation

Objective — Minimize:

```
\frac{1}{2}||w||^2 + C \sum_{i=1}^{n}(\xi_i + \xi_i^*)
```

Subject to:

- y_i - (w * x_i + b) <= epsilon + xi_i
- (w * x_i + b) - y_i <= epsilon + xi_i*
- xi_i, xi_i* >= 0

Where:

- **epsilon**: tube half-width — tolerance for error
- **xi (slack variable)**: penalty for points outside the tube
- **C**: how hard we penalize violations outside the tube

SVR Parameters

Parameter	Effect	Small Value	Large Value
epsilon	Tube width	Narrow tube, more support vectors	Wide tube, fewer support vectors
C	Penalty for violations	Smoother curve (underfitting)	Tighter fit to data (overfitting)
kernel	Decision boundary shape	linear = straight line	rbf = curved

Python Implementation

```
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
import numpy as np
import matplotlib.pyplot as plt

# Generate House Price Data
np.random.seed(42)
size = np.random.uniform(500, 3000, 100).reshape(-1, 1)  # sq ft
price = 50000 + 150 * size.ravel() + np.random.normal(0, 20000, 100)

# Scale features (critical for SVR!)
scaler_X = StandardScaler()
scaler_y = StandardScaler()
X_scaled = scaler_X.fit_transform(size)
y_scaled = scaler_y.fit_transform(price.reshape(-1, 1)).ravel()

# Train SVR
svr_rbf = SVR(kernel='rbf', C=100, epsilon=0.1, gamma='scale')
svr_rbf.fit(X_scaled, y_scaled)

# Predict and inverse transform back to original scale
y_pred_scaled = svr_rbf.predict(X_scaled)
y_pred = scaler_y.inverse_transform(y_pred_scaled.reshape(-1, 1)).ravel()
```

Visualizing the Epsilon Tube

```
sort_idx = np.argsort(size.ravel())
X_sorted = size.ravel()[sort_idx]
y_actual_sorted = price[sort_idx]
y_pred_sorted = y_pred[sort_idx]

plt.figure(figsize=(10, 6))
plt.scatter(X_sorted, y_actual_sorted, alpha=0.5, label='Actual Prices')
plt.plot(X_sorted, y_pred_sorted, 'r-', linewidth=2, label='SVR Prediction')

epsilon_original = scaler_y.scale_[0] * svr_rbf.epsilon * 1000
plt.fill_between(X_sorted,
                 y_pred_sorted - epsilon_original,
                 y_pred_sorted + epsilon_original,
                 alpha=0.3, color='green', label='epsilon-tube')

plt.xlabel('House Size (sq ft)')
plt.ylabel('Price ($)')
plt.title('SVR with Epsilon Tube')
plt.legend()
plt.show()
```

SVR vs Linear Regression

Aspect	Linear Regression	SVR
Sensitive to outliers	Yes (all points affect fit)	**No** (only boundary points matter)
Handles non-linearity	No (by default)	**Yes** (kernel trick)
Training speed	Fast	Slower for large datasets
Interpretability	High	Low
Best for	Simple linear data	Complex, noisy data with outliers

Kernel Comparison for SVR

```
kernels = ['linear', 'rbf', 'poly']
for kernel in kernels:
    svr = SVR(kernel=kernel, C=100, epsilon=0.1)
    svr.fit(X_scaled, y_scaled)
    score = svr.score(X_scaled, y_scaled)  # R-squared score
    print(f'{kernel:8s}: R-squared = {score:.4f}')
```

R-squared: Measures how much variance the model explains. R-squared = 1.0 is perfect; R-squared = 0 means the model is no better than predicting the mean.

7. Real World Applications + SVM Cheat Sheet

Where SVM is Used in the Real World

Industry	Problem	Why SVM Works
Healthcare	Cancer detection, disease diagnosis	High accuracy on small-to-medium medical datasets
Finance	Credit risk scoring, fraud detection	Handles high-dimensional financial features well
Image Recognition	Face detection, digit recognition	Effective with pixel feature vectors
Text Classification	Spam filtering, sentiment analysis	Excellent in high-dimensional text (TF-IDF) spaces
Bioinformatics	Gene expression classification	Small samples, many features — SVM's sweet spot
Remote Sensing	Satellite image land-use classification	Multi-class problems with complex boundaries

Real-World Example Walkthroughs

Example 1 — Email Spam Filter: Imagine each email is a point in 10,000-dimensional space (one dimension per word). SVM draws a hyperplane separating "spam" from "not spam." Only the most borderline emails (support vectors) define this boundary. This is why SVM is used in production email systems — it generalizes from relatively few training examples.

Example 2 — Handwriting Recognition: Each digit image (say 28x28 pixels = 784 features) is mapped to a 784-dimensional space. The RBF kernel finds non-linear boundaries between digit classes (e.g., the difference between '1' and '7' is subtle). SVM with RBF kernel achieves >99% accuracy on MNIST with proper tuning.

Example 3 — Stock Market: Given 50 financial indicators (RSI, MACD, moving averages, etc.), SVM predicts whether a stock will go Up or Down tomorrow. The kernel trick captures non-linear relationships between indicators.

When to Use SVM vs Other Algorithms

Situation	Best Algorithm	Reason
Small dataset (<10K samples), many features	**SVM**	Generalizes well, kernel trick
Large dataset (>100K samples)	Logistic Regression / Neural Net	SVM training is $O(n^3)$, too slow
Need probability scores	Logistic Regression or SVM with `probability=True`	SVM probabilities via Platt scaling
Interpretability required	Logistic Regression / Decision Tree	SVM is a black box
Noisy data with outliers	SVM (soft margin)	C parameter absorbs outliers
Non-linear, complex patterns	SVM (RBF kernel)	Kernel trick handles non-linearity

Complete SVM Cheat Sheet

Imports:

```
from sklearn.svm import SVC, SVR, LinearSVC
from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

Quick Templates:

```
# SVM Classification Pipeline (Best Practice)
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf', C=10, gamma='scale'))
])
pipeline.fit(X_train, y_train)

# SVM Regression Pipeline
pipeline_svr = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(kernel='rbf', C=100, epsilon=0.1))
])

# GridSearchCV Template
param_grid = {'C': [0.1, 1, 10, 100], 'gamma': ['scale', 'auto', 0.001,
0.01]}
grid = GridSearchCV(SVC(kernel='rbf'), param_grid, cv=5, scoring='f1')
grid.fit(X_train_scaled, y_train)
best = grid.best_estimator_

# Save Model
import joblib
joblib.dump(pipeline, 'svm_model.pkl')
loaded_model = joblib.load('svm_model.pkl')
```

Key Parameters Reference:

Parameter	Values	Effect
`kernel`	'linear', 'rbf', 'poly', 'sigmoid'	Decision boundary shape
`C`	0.001 → 1000	Regularization (lower = smoother)
`gamma`	'scale', 'auto', float	RBF/Poly bandwidth
`degree`	2, 3, 4	Polynomial degree (only for 'poly')
`epsilon`	0.001 → 1.0	SVR tube width
`probability`	True/False	Enable predict_proba()

Decision Rules:

- Always scale before SVM

- Start with RBF kernel + GridSearchCV
- Use `probability=True` for ROC curves
- Use `scoring='recall'` for medical/safety problems
- Use `LinearSVC` for text classification (faster on high-dim sparse data)

8. SVM Capstone Project

Project Overview

This capstone consolidates everything learned about SVM into a complete, production-ready machine learning pipeline applied to the Breast Cancer Wisconsin dataset.

Part A: Kernel War — Which Kernel Wins?

Goal: Systematically compare all kernel types and pick the best one.

```
from sklearn.svm import SVC
from sklearn.metrics import classification_report
import pandas as pd

# Define kernels to test
kernel_configs = [
    {'name': 'Linear', 'kernel': 'linear', 'C': 1},
    {'name': 'RBF (C=1)', 'kernel': 'rbf', 'C': 1},
    {'name': 'RBF (C=10)', 'kernel': 'rbf', 'C': 10},
    {'name': 'Poly d=2', 'kernel': 'poly', 'degree': 2, 'C': 1},
    {'name': 'Poly d=3', 'kernel': 'poly', 'degree': 3, 'C': 1},
]

results = []
for config in kernel_configs:
    name = config.pop('name')
    model = SVC(**config)
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    report = classification_report(y_test, y_pred, output_dict=True)
    results.append({
        'Kernel': name,
        'Accuracy': report['accuracy'],
        'Recall (Malignant)': report['0']['recall'],
```

```
        'F1 (Weighted)': report['weighted avg']['f1-score']
    })

df_results = pd.DataFrame(results)
print(df_results.sort_values('Recall (Malignant)', ascending=False))
```

Part B: Feature Analysis — What Makes a Tumor Dangerous?

Goal: Find which features matter most by analyzing support vectors.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Correlation Heatmap of top features
corr_matrix = pd.DataFrame(X_train, columns=data.feature_names).corr()

plt.figure(figsize=(12, 8))
sns.heatmap(corr_matrix.iloc[:10, :10],
            annot=True, fmt='.2f',
            cmap='RdYlGn', center=0,
            linewidths=0.5)
plt.title('Feature Correlation Heatmap (Top 10 Features)')
plt.tight_layout()
plt.show()

# Feature importance via permutation
from sklearn.inspection import permutation_importance
best_model.probability = False  # speed up
perm_imp = permutation_importance(best_model, X_test_scaled, y_test,
                                  n_repeats=10, random_state=42)
feat_imp_df = pd.DataFrame({
    'Feature': data.feature_names,
    'Importance': perm_imp.importances_mean
}).sort_values('Importance', ascending=False)
print(feat_imp_df.head(10))
```

Part C: Feature Selection — Can Fewer Features Work Better?

Goal: Test if removing weak features improves performance.

```
from sklearn.feature_selection import SelectKBest, f_classif

# Select top K features
for k in [5, 10, 15, 20, 30]:
    selector = SelectKBest(f_classif, k=k)
    X_train_sel = selector.fit_transform(X_train_scaled, y_train)
    X_test_sel = selector.transform(X_test_scaled)

    model_sel = SVC(kernel='rbf', C=10, gamma='scale')
    model_sel.fit(X_train_sel, y_train)
    score = model_sel.score(X_test_sel, y_test)
    print(f'Top {k:2d} features: Accuracy = {score:.4f}')
```

Part D: Deploy-Ready Pipeline

Goal: Package the complete model for production use.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
import joblib

# Build complete pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf', probability=True))
])

# Tune on raw (unscaled) data - pipeline handles scaling internally
param_grid = {
    'svm__C': [1, 10, 100],
    'svm__gamma': ['scale', 'auto', 0.01]
}
grid = GridSearchCV(pipeline, param_grid,
                    scoring='recall', cv=5)
grid.fit(X_train, y_train)  # pass raw X_train!

best_pipeline = grid.best_estimator_
y_pred_final = best_pipeline.predict(X_test)  # pass raw X_test!

# Save for deployment
```

```
joblib.dump(best_pipeline, 'breast_cancer_svm.pkl')
print('Model saved to breast_cancer_svm.pkl')

# To load and use later:
# model = joblib.load('breast_cancer_svm.pkl')
# prediction = model.predict(new_patient_data)  # no scaling needed!
```

What We Learned in This Course

Concept	Key Takeaway	Real-World Use
SVM Core Idea	Maximum margin classifier using support vectors	Any binary/multi-class classification
Hyperplane & Margin	Decision boundary + buffer zone; wider = better generalization	Understanding model confidence
Kernel Trick	Map data to higher dimensions without explicit computation	Non-linear pattern detection
C Parameter	Trade-off between margin width and misclassification	Regularization in noisy datasets
Gamma Parameter	Controls RBF kernel influence radius	Prevents overfitting
SVR	Regression with epsilon tube; ignores small errors	Price prediction, demand forecasting
GridSearchCV	Systematic hyperparameter search	Production model optimization
Recall vs Precision	Context-dependent metric choice	Critical in medical/safety systems
Pipelines	Encapsulate preprocessing + model	Clean, production-ready code
joblib	Save/load trained models	Model deployment

How SVM Fits Into Your ML Journey

You now know three powerful classifiers:

1. **Logistic Regression** — Probabilistic, interpretable, fast. Use when you need probability scores and explanations.
2. **K-Nearest Neighbors** — Intuitive, no training phase, instance-based. Use for small datasets when decision boundaries are complex.
3. **Support Vector Machine** — Maximum margin, kernel trick, robust to outliers. Use when precision matters and datasets are small-to-medium.

Each algorithm has its strengths. A skilled ML practitioner chooses the right tool for the right problem — and now you can do that.